Homework 10

Author: Manushri Pendekanti (mp79)

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.metrics import confusion_matrix, accuracy_score
%matplotlib inline

from rlt_fallback import RLT_cla

# Homework asks for the RLT package; use sklearn random forests if `rlt` is
# pip install git+https://github.com/teazrq/RLT-Python.git  (may need OpenMP
```

Question 1: Bias-variance trade-off simulation

Regression with 5 correlated predictors (mean 0, var 1, corr 0.3); $Y = X_1 + X_3 + X_5 + \varepsilon$, $\varepsilon \sim N(0,1)$.

Use **500-tree** ordinary random forests (`RLT` / `rlt`), reinforcement off. **100 repetitions**; **200 fixed test** points; **200 training** obs per rep (same training data for all tuning settings within a rep); **200 bootstrap** draws per tree.

In [ ]:
```
# Question 1 setup (Python translation of R MASS::mvrnorm + training lists)
rng = np.random.default_rng(123)  # picked seed 123

Sigma = np.full((5, 5), 0.3)
np.fill_diagonal(Sigma, 1.0)

x_test = rng.multivariate_normal(np.zeros(5), Sigma, size=200)
# Columns 1, 3, 5 in R -> indices 0, 2, 4 in Python
f_test = x_test[:, 0] + x_test[:, 2] + x_test[:, 4]

x_train = []
y_train = []

for r in range(100):
    x = rng.multivariate_normal(np.zeros(5), Sigma, size=200)
    y = x[:, 0] + x[:, 2] + x[:, 4] + rng.normal(size=200)
    x_train.append(x)
    y_train.append(y)
```

Question 1 — Part a, first variation

(Your written answer.)

In [ ]:
```
# Part 1a – variation 1: mtry = 3, vary node size
rng = np.random.default_rng(124)

node_sizes = [1, 5, 10, 20, 30]
node_predictions = []
node_bias2 = []
node_variance = []

for node_size in node_sizes:
    predictions = np.zeros((200, 100))

    for r in range(100):
        fit = RandomForestRegressor(
            n_estimators=500,
            max_features=3,
            min_samples_leaf=node_size,
            max_samples=200,
            bootstrap=True,
            random_state=124 + r,
            n_jobs=-1,
        )
        fit.fit(x_train[r], y_train[r])
        predictions[:, r] = fit.predict(x_test)

    node_predictions.append(predictions)

    bias2 = (predictions.mean(axis=1) - f_test) ** 2
    variance = predictions.var(axis=1, ddof=1)

    node_bias2.append(bias2.mean())
    node_variance.append(variance.mean())

node_results = pd.DataFrame({
    "nodesize": node_sizes,
    "bias_squared": node_bias2,
    "variance": node_variance,
})
print(node_results)

plt.figure(figsize=(7, 4))
plt.plot(node_sizes, node_bias2, "b-o", label="Squared bias")
plt.plot(node_sizes, node_variance, "r-s", label="Variance")
plt.xlabel("Node size")
plt.ylabel("Average squared bias and variance")
plt.legend(loc="upper left")
plt.title("Fixed mtry = 3")
plt.show()
```

In [ ]:
```
# Part 1a – variation 2: node size = 10, vary mtry
rng = np.random.default_rng(125)

mtry_values = [1, 2, 3, 4, 5]
mtry_predictions = []
```

```
mtry_bias2 = []
mtry_variance = []

for mtry in mtry_values:
    predictions = np.zeros((200, 100))

    for r in range(100):
        fit = RandomForestRegressor(
            n_estimators=500,
            max_features=mtry,
            min_samples_leaf=10,
            max_samples=200,
            bootstrap=True,
            random_state=125 + r,
            n_jobs=-1,
        )
        fit.fit(x_train[r], y_train[r])
        predictions[:, r] = fit.predict(x_test)

    mtry_predictions.append(predictions)

    bias2 = (predictions.mean(axis=1) - f_test) ** 2
    variance = predictions.var(axis=1, ddof=1)

    mtry_bias2.append(bias2.mean())
    mtry_variance.append(variance.mean())

mtry_results = pd.DataFrame({
    "mtry": mtry_values,
    "bias_squared": mtry_bias2,
    "variance": mtry_variance,
})
print(mtry_results)

plt.figure(figsize=(7, 4))
plt.plot(mtry_values, mtry_bias2, "b-o", label="Squared bias")
plt.plot(mtry_values, mtry_variance, "r-s", label="Variance")
plt.xlabel("mtry")
plt.ylabel("Average squared bias and variance")
plt.legend(loc="upper left")
plt.title("Fixed node size = 10")
plt.show()
```

Question 1 — Part b

a.) a larger node size produces smaller trees which have a smaller amout of splits and may end up underfitting

b.) a larger mtry can reduce bias because each time tree splits it get more predicators which can end up choosing a better predicator but because of this it may end up choosing very similar splits and can end up becoming correlated

c.) I would expect it to shift toward smaller values because of smaller because a smaller node size will reduce the amount of splitting and reduce number or make smaller trees.

Question 2: Tuning random forests for MNIST

Data: `data/mnist-first2000.csv.gz` (`Digit` + 784 pixels).

Training: first 1,000 rows, digits **1, 6, 7** only. Test: remaining 1,000 rows, same digits. PCA on training pixels (center, do not scale); use **first 20 PCs**; project test with training PCA.

1,000-tree RLT classification forests; bootstrap sample size = training size; ordinary RF (reinforcement off).

In [ ]:
```
# Question 2 setup
mnist = pd.read_csv("data/mnist-first2000.csv.gz", compression="gzip")

train = mnist.iloc[:1000].copy()
test = mnist.iloc[1000:2000].copy()

train = train[train["Digit"].isin([1, 6, 7])].copy()
test = test[test["Digit"].isin([1, 6, 7])].copy()

pixel_cols = [c for c in mnist.columns if c != "Digit"]
x_train = train[pixel_cols]
x_test = test[pixel_cols]
y_train = train["Digit"].astype(int)
y_test = test["Digit"].astype(int)

# PCA on training pixels: center only, do not scale
pca = PCA(n_components=20)
train_pc = pca.fit_transform(x_train)
test_pc = pca.transform(x_test)

print("Training shape:", train_pc.shape)
print("Test shape:", test_pc.shape)
```

Question 2 — Part a

(Your written answer.)

In [ ]:
```
# Question 2 — Part a: default random forest
default_fit = RLT_cla(
    n_estimators=1000,
    resample_prob=1,
    resample_replace=True,
    random_state=123,
    n_jobs=1
)

default_fit.fit(train_pc, y_train)
```

```
print("Default OOB misclassification rate:", default_fit.oob_error_)
```

Question 2 — Part b

(Your written answer.)

In [ ]:
```
# Question 2 – Part b: tune mtry and node size
results = []
models = []

for mtry in [1, 4, 10, 20]:
    for node_size in [1, 5, 10, 20, 30]:
        fit = RLT_cla(
            n_estimators=1000,
            mtry=mtry,
            min_samples_leaf=node_size,
            resample_prob=1,
            resample_replace=True,
            importance="permute",
            random_state=123,
            n_jobs=1
        )

        fit.fit(train_pc, y_train)

        results.append({
            "mtry": mtry,
            "Node size": node_size,
            "OOB error": float(fit.oob_error_)
        })

        models.append(fit)


results = pd.DataFrame(results)

print("OOB error for each setting:")
print(results.to_string(index=False))


best = results["OOB error"].idxmin()
best_fit = models[best]

best_mtry = int(results.loc[best, "mtry"])
best_node = int(results.loc[best, "Node size"])
best_error = results.loc[best, "OOB error"]

print("\nSelected mtry:", best_mtry)
print("Selected node size:", best_node)
print(f"Selected OOB misclassification rate: {best_error:.4f}")
print(f"Selected OOB misclassification percentage: {100 * best_error:.2f}%")


importance_results = pd.DataFrame({
```

```
    "PC": [f"PC{i}" for i in range(1, 21)],
    "Importance": np.asarray(best_fit.feature_importances_).ravel()
})

print("\nOOB permutation importance:")
print(importance_results.to_string(index=False))


ranked = importance_results.sort_values(
    "Importance", ascending=False
)

top_three = ", ".join(ranked["PC"].iloc[:3])


print("\nComment:")
print(
    f"The selected forest used mtry = {best_mtry} and "
    f"node size = {best_node}, with an OOB misclassification "
    f"rate of {100 * best_error:.2f}%. "
    f"The three PCs with the highest permutation importance "
    f"were {top_three}. "
    "positive values indicate that"
    "the PC increased OOB error more. Values at zero suggest "
    "little cause to this forest. Earlier PCs explain "
    "more variation, but they are not the "
    "most useful for differentiating digits."
)
```

Question 2 — Part c

(Your written answer.)

In [ ]:

```
# Question 2 — Part c: test-set evaluation

y_test = test["Digit"].to_numpy()

test_prediction = best_fit.predict(test_pc)

test_error = np.mean(test_prediction != y_test)
print("Test misclassification rate:", test_error)

matrix = confusion_matrix(
    y_test,
    test_prediction,
    labels=[1, 6, 7]
)

matrix = pd.DataFrame(
    matrix,
    index=["Actual 1", "Actual 6", "Actual 7"],
    columns=["Predicted 1", "Predicted 6", "Predicted 7"]
)
```

```
print(matrix)
```

Question 3: Planning a random-forest analysis skill

Discuss how you would write a skill (no complete skill file required).

Question 3 — Part a

(One paragraph: balancing standard instructions vs. personal preferences.)

How I would write the skill is first allow it to actually look into the data and understand it and then other basic things like fit the forests etc. I would keep some specfic rules fixed like ways I would prevent any of the test data leakage. Based on the dataset allow skill to choose between different classfication etc and different kinds of evualation metrics. Also I would add my preferences like always writing simple easy to understand code even if it longer and results are easy to follow for any with well organization. In addition I would give all of this rules in easy step by step flow for AI to easy understand.

Question 3 — Part b

(Ten ordered instructions, basic → personal preference.)

1.) First get the what is response and predicators and pick between classfication and regression

2.) clean the dataset and find way to handle like missing values and wrong data types

3.) User givies a certain train/test split follow it and perform

4.)import packages and set seed

5.) first fit a baseline random forest

6.) then we will tune mtry and print all different combinations and errors

7.)selected model performance and OOB importance

8.) display in graphs and tables results

9.) always use easy to understand simple code without overcompliating

10.) Write a neat summary and to summarixe everyhting in graphs and tables